

FlowDPG: Deterministic Policy Gradient on Flow Matching Policies for Real-World Manipulation

Kexin Shi^{1,2} Junyao Shi^{1,3} Poorvi Hebbar¹ Zhuolun Zhao¹
 Tarun Amarnath¹ Yifan Su¹ Shikhar Bahl^{1†} Deepak Pathak^{1,2†}
¹Skild AI ²Carnegie Mellon University ³University of Pennsylvania

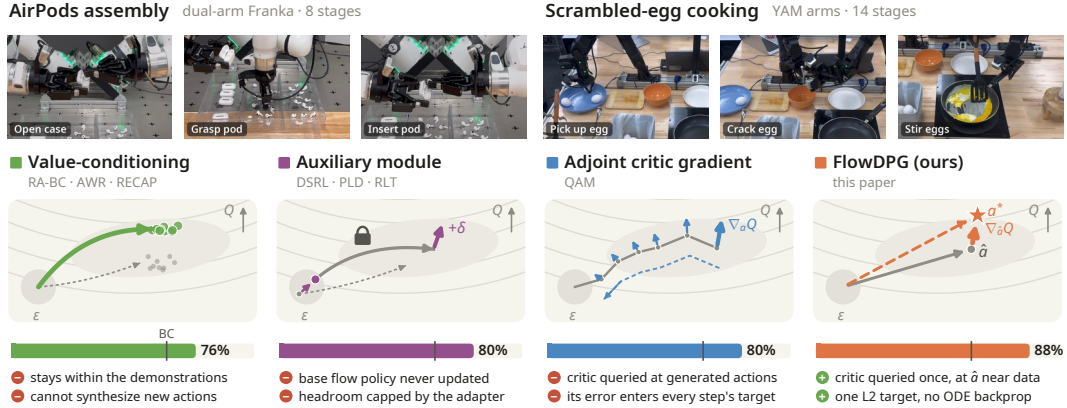


Figure 1: **Top:** the two real-robot tasks. **Bottom:** how three families of RL fine-tuning methods (value-conditioning, auxiliary modules, and adjoint critic gradients) update a flow policy, their main limitations, and how FlowDPG avoids them. Bars: best offline AirPods success per family.

Abstract: Real-world reinforcement learning for robotic manipulation remains challenging, and this difficulty is amplified for flow matching policies: policy gradients must be backpropagated through time (BPTT) along the multi-step ODE that maps noise to actions, which is computationally prohibitive and numerically fragile. We propose **FlowDPG**, a DDPG-style method for flow matching policies that bypasses BPTT entirely. FlowDPG evaluates the critic gradient at a one-step estimate of the clean action and distills the resulting correction into the velocity field at training time, leaving multi-step inference unchanged. Intuitively, it combines two complementary vectors: the demonstration-driven velocity that keeps the action feasible, and the critic-driven correction that steers it toward higher value. Our contributions are threefold: (1) a BPTT-free distillation framework for stable DDPG-style improvement of flow matching policies, (2) a formal connection between the FlowDPG update and the vanilla deterministic policy gradient via three explicit approximations, and (3) real-world validation on two long-horizon tasks on different robots. FlowDPG reaches 92% end-to-end success on dual-arm AirPods assembly with Franka arms and an 86% rubric score on scrambled-egg cooking with YAM arms, substantially outperforming recent RL methods. Videos and more results: <https://flowdpg.github.io>.

Keywords: Flow Matching, Offline-to-Online RL, Real-World Manipulation

[†]Equal advising.

1 Introduction

Despite the successes of large-scale supervised models in robotics, imitation learning alone remains insufficient for long-horizon, contact-rich tasks. Collecting robot data at the scale and signal-to-noise ratio of the datasets used to train large language models is prohibitively expensive, and even with abundant high-quality teleoperation, a supervised policy inherits its demonstration distribution and degrades sharply in settings the data do not cover. On a long-horizon task, even rare failures at each stage compound into frequent end-to-end failure. This motivates letting the robot learn from its own experience. Real-world reinforcement learning, which has driven much of the recent progress in post-training large language models, is a natural candidate, as it can push the policy beyond the demonstration data.

Recent state-of-the-art manipulation policies have converged on complex generative approaches such as flow matching [1] and diffusion [2] action heads, which produce expressive multi-modal action chunks through a multi-step denoising ODE. For these policies, the standard off-policy actor-critic update [3] propagating the critic gradient $\nabla_a Q$ back through the actor requires backpropagation through the entire denoising ODE. This is computationally prohibitive, since the cost grows linearly with the number of solver steps. It is also numerically fragile because the chain of Jacobians compounds gradient explosion and vanishing pathologies, particularly severe when the velocity field is not yet well-trained. As a result, expressive flow policies and value-driven RL have been hard to combine without either discarding the critic gradient or replacing the multi-step policy with a one-step actor.

We propose **FlowDPG**, an off-policy actor-critic method designed for flow matching policies that obtains a stable critic-gradient update without backpropagating through the denoising ODE. The key insight is that the clean action x_1 can be estimated from any intermediate noisy action x_t by a single forward pass of the velocity predictor, as done for diffusion policies in EDP [4]. We evaluate the critic gradient at this projected clean action, combine it with the demonstration-driven velocity to form a value-improved target, and distill the result back into the velocity field via L2 regression. Intuitively, FlowDPG composes two complementary vectors: the demonstration-driven velocity that keeps the action feasible, and the critic-driven correction that steers it toward higher value. Inference is unchanged from standard flow matching, a pure ODE solve with no deployment-time critic queries. A core property of FlowDPG is that its update direction can be derived from the vanilla DPG gradient on a flow policy under three explicit, controllable approximations, detailed in Section 3.2. This places the method on a transparent theoretical footing relative to the classical DDPG literature, in contrast to recent critic-gradient approaches that rely on more abstract stochastic-control formulations [5].

We validate FlowDPG on two long-horizon real-world tasks on different robots. On contact-rich, dual-arm AirPods assembly with millimeter-level insertion tolerance, where a single policy runs all eight stages end-to-end from raw observations, FlowDPG reaches 92% end-to-end success, 28 points above the BC base. On 14-stage scrambled-egg cooking with tool use and fluid handling, offline training alone raises the rubric score from 64.2% to 85.8%. It leads the closest baseline, QAM, on both tasks in a repeated evaluation, and online fine-tuning improves recovery from human disturbances. In summary, our contributions are (1) a BPTT-free distillation framework for stable DDPG-style improvement of flow matching policies, (2) a formal connection between FlowDPG and vanilla DPG via three explicit approximations, and (3) real-world validation on two long-horizon tasks across two robot platforms, with ablations supporting each design choice.

2 Related Work

Generalist manipulation policies and generative action heads. Large-scale robot datasets [6, 7, 8, 9] have enabled generalist policies, from multi-task transformers [10, 11, 12] to vision-language-action models [13, 14, 15, 16, 17] built on large vision-language models [18, 19, 20, 21, 22]. At the action level, policies have moved to expressive generative heads: action-chunking transformers [23], diffusion policies [2, 24, 25], and flow matching [1, 14, 17], the actor class we focus on.

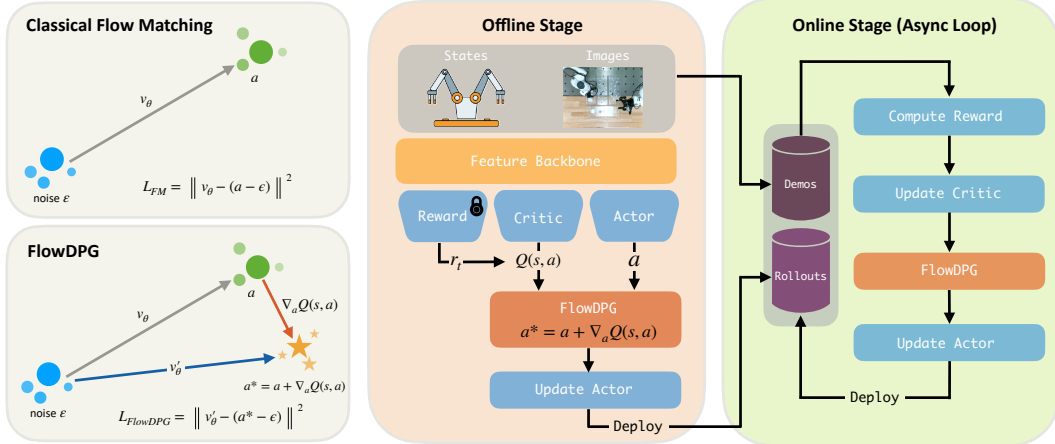


Figure 2: **FlowDPG overview.** **Left:** classical flow matching (top) transports noise to a demonstration-feasible action a via v_θ ; FlowDPG (bottom) adds a critic-driven correction along $\nabla_a Q$ to reach a value-improved target a^* (schematic; Sec. 3.2 queries the critic at the one-step estimate $\hat{a}$ with an adaptive step), then distills the resulting velocity v'_θ back into the flow field. **Middle:** in the offline stage, a shared visual backbone feeds a frozen reward predictor, a critic, and a flow matching actor; the FlowDPG block combines the actor output with the critic gradient into a^* , which is distilled back into the actor. **Right:** in the online stage, the actor is rolled out on the real robot, and the critic and actor are asynchronously updated from a replay buffer mixing demonstrations with fresh rollouts.

RL fine-tuning of expressive manipulation policies. Recent work has explored several ways to improve imitation-trained manipulation policies with reward signals. One family discards the critic’s action gradient and uses only scalar value or advantage information: RA-BC [26] reweights the behavior-cloning objective by progress estimates, AWR [27] fits the policy to advantage-weighted demonstrations, and RECAP [28] conditions the policy on advantage as an additional input. A second family keeps the base policy frozen and learns a separate adaptation module: DSRL [29] performs RL in the latent noise space of the denoiser, PLD [30] learns a single-step residual and distills it back through supervised fine-tuning, and RLT [31] attaches a lightweight actor-critic head to a compact token extracted from the frozen base policy. The closest family exploits the critic’s action gradient directly: Q-score matching [32] relates the score of an optimal policy to gradients of the Q-function, while QAM [5, 33] converts $\nabla_a Q$ into step-wise supervision through an auxiliary adjoint flow. Diffusion-QL [34] backpropagates Q through the full denoising chain, and EDP [4] replaces the chain with a one-step estimate of the clean action. FlowDPG evaluates the critic gradient at the analogous flow estimate, but normalizes the step per sample, distills it as an L2 target, and derives the update as an explicit approximation of vanilla DPG. On-policy methods avoid BPTT by making each denoising step a Gaussian policy with a tractable likelihood: DPPO [35], ReinFlow [36], and Flow-GRPO [37] then apply PPO- or GRPO-style updates, at the cost of on-policy data. A parallel line avoids BPTT by changing the actor class: FQL [38] distills the flow into a one-step actor trained against Q , QC-FQL [39] extends it to action chunks, SSCQL [40] and OFQL [41] train a single network for one-step generation, and the concurrent DriftQL [42] uses a one-step drift policy. All of them deploy a single-step generator. In the official FQL setting, fresh noise is drawn at every control step and a single action is executed, so closed-loop feedback absorbs the stochasticity. When a one-step actor is executed in action chunks on hardware, a fresh sample at each chunk boundary instead selects a different trajectory, making chunk transitions jerky and undeployable at millimeter tolerance, while fixing the noise removes the multimodality the generative head was meant to provide. FlowDPG instead improves the multi-step flow policy in place. For value estimation, we use IQL [43] with twin critics [44] and a stage-aware reward predictor [26] as the dense reward source.

3 Method

Our method has two coupled components on top of a flow matching policy (Figure 2): a twin-critic IQL value estimator (Sec. 3.1), and **FlowDPG**, which distills the deterministic policy gradient (DPG) [45, 3] into the flow matching velocity field at training time (Sec. 3.2).

3.1 Value Estimation with Twin-Critic IQL

We adopt Implicit Q-Learning (IQL) [43] with twin critics in the style of TD3 [44]. IQL replaces the $\max_a Q(s, a)$ bootstrap target, which queries the critic on potentially out-of-distribution actions, with a value function $V_\psi(s)$ that estimates an upper-expectile in-distribution action-value at s , eliminating extrapolation error during value learning. Twin critics Q_{ϕ_1}, Q_{ϕ_2} further mitigate Q-value overestimation from OOD actions and TD-bootstrap noise; we take their minimum wherever a single Q -estimate is needed, and pair each critic with an EMA target network used in the value loss. V_ψ is trained by expectile regression of the twin-critic minimum on dataset actions,

$$\mathcal{L}_V(\psi) = \mathbb{E}_{\mathcal{D}}[\rho_\tau(\min_{i=1,2} Q_{\phi_i}(s, \mathbf{a}) - V_\psi(s))], \quad \rho_\tau(u) = |\tau - \mathbb{I}[u < 0]| \cdot u^2, \quad (1)$$

with expectile $\tau > 0.5$ pulling $V_\psi(s)$ toward the upper tail of the action-value distribution. The critics are then bootstrapped from V_ψ :

$$y_Q = r_t + \gamma^H V_\psi(s_{t+H}), \quad \mathcal{L}_Q(\phi_i) = \mathbb{E}_{\mathcal{D}}[(Q_{\phi_i}(s_t, \mathbf{a}_t) - y_Q)^2], \quad (2)$$

again avoiding extrapolation to unseen actions. Chunk-level rewards r_t come from SARM [26], a frozen stage-aware reward predictor that outputs a stage label $z(o_t)$ and a within-episode progress $\phi(o_t) \in [0, 1]$, combined into a composite chunk reward of three terms:

$$r_t = \underbrace{\phi(o_{t+H}) - \phi(o_t)}_{\text{progress}} + \underbrace{\beta_{\text{stage}} \cdot \mathbb{I}[z(o_{t+H}) > z(o_t)]}_{\text{stage_transition}} + \underbrace{r_t^{\text{term}}}_{\text{terminal}}. \quad (3)$$

The progress term provides dense per-step shaping, the indicator term issues a fixed bonus β_{stage} at each predicted stage advance, and the terminal term encodes whole-episode success; we visualize an example reward and value curve in Appendix C.

3.2 Policy Extraction via FlowDPG

A flow matching policy transports noise x_0 to a feasible action x_1 within the demonstration support, but the resulting action is not necessarily optimal under the critic. FlowDPG combines *two vectors* (Figure 2, left): the demonstration-driven velocity for feasibility, and a critic-driven correction along $\nabla_a Q$ in the spirit of DDPG [3, 45] for value improvement. The combined target is distilled back into v_θ via a single L2 regression, avoiding the obvious alternative of backpropagating $\nabla_a Q$ through the full ODE, which is computationally prohibitive and numerically fragile. FlowDPG costs only one extra forward pass and one critic-gradient evaluation per update, and leaves inference as a pure flow ODE solve.

Q-Gradient. The clean action x_1 can be estimated from any intermediate x_t by a single forward pass of v_θ :

$$\hat{a} := x_t + (1 - t) \cdot v_\theta(x_t, t, s). \quad (4)$$

For an accurate v_θ , $\hat{a}$ equals $\mathbb{E}[x_1 \mid x_t]$ and approaches x_1 as $t \rightarrow 1$; the consistency regularizer below reduces its fitting error. We compute $\hat{a}$ under `no_grad` and evaluate the critic gradient there:

$$g = \nabla_{\hat{a}}[\min_{i=1,2} Q_{\phi_i}(s, \hat{a})], \quad (5)$$

with stop-grad on the critic parameters.

Adaptive shift. The magnitude of g varies considerably across states and training steps, destabilizing updates if used as-is. We normalize the step to the local velocity scale:

$$\Delta = \alpha \cdot \frac{\|u_t\|}{\|g\| + \varepsilon} \cdot g, \quad (6)$$

with $\alpha > 0$ a base guidance scale and ε guarding against vanishing gradients.

Distilled flow target. Summing $\hat{a}$ and Δ gives the Q-improved clean action, and subtracting noise gives the velocity target:

$$a^* = \hat{a} + \Delta, \quad u_t^* = a^* - x_0. \quad (7)$$

The policy is regressed against u_t^* alongside a behavior-cloning anchor:

$$\mathcal{L}_{\text{FlowDPG}}(\theta) = \lambda \cdot \underbrace{\|v_\theta(x_t, t, s) - u_t^*\|^2}_{\mathcal{L}_{\text{distill}}} + (1 - \lambda) \cdot \underbrace{\|v_\theta(x_t, t, s) - u_t\|^2}_{\mathcal{L}_{\text{BC}}}, \quad (8)$$

with λ warming up linearly from 0 over N_{warmup} steps so that the unreliable early critic does not dominate.

Consistency regularizer. The reliability of $\hat{a}$ is critical: a projection off the clean-action manifold leaves the critic queried OOD and g unreliable. Since the standard flow loss only supervises local velocity, we add, with weight μ_{cons} ,

$$\mathcal{L}_{\text{cons}}(\theta) = \mathbb{E}[\|\hat{a} - x_1\|^2]. \quad (9)$$

Substituting $x_t = (1 - t)x_0 + tx_1$ into Eq. 4 gives $\hat{a} - x_1 = (1 - t)(v_\theta - u_t)$, so $\mathcal{L}_{\text{cons}}$ reduces to a velocity-error penalty weighted by $(1 - t)^2$, emphasizing small- t samples where the projection error is most amplified.

Relationship to vanilla DPG. A vanilla DPG [45, 3] update on a flow matching policy treats the ODE terminus x_1 as a deterministic actor and, by the chain rule (Appendix A), yields

$$\nabla_\theta \mathcal{L}_{\text{DPG}} = -\nabla_a Q(s, x_1) \cdot \nabla_\theta x_1 = -\int_0^1 g^\top \Phi(1, t) \cdot \nabla_\theta v_\theta(x_t, t, s) dt, \quad (10)$$

where $\Phi(1, t) := \partial x_1 / \partial x_t$ is the ODE sensitivity matrix. Computing Φ requires backpropagating through all T ODE steps with $O(T)$ memory and well-known gradient explosion/vanishing pathologies, particularly severe when v_θ is not yet well-trained. The FlowDPG gradient, by contrast, comes from differentiating $\mathcal{L}_{\text{FlowDPG}}$: after warmup, the only g -dependent term is the distillation loss; treating g as a detached constant, its part of the gradient is

$$[\nabla_\theta \mathcal{L}_{\text{FlowDPG}}]_g = -2\lambda\alpha\|u_t\| \cdot \hat{g}(\hat{a}) \cdot \nabla_\theta v_\theta(x_t, t, s), \quad (11)$$

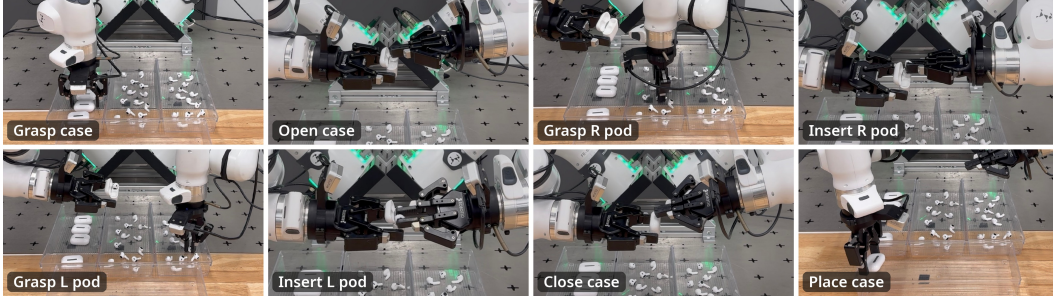
with $\hat{g}(\hat{a}) := g/\|g\|$. This recovers Eq. 10 under three explicit approximations: (i) the critic gradient is evaluated at the one-step estimate $\hat{a}$ rather than at the ODE terminus, collapsing $\Phi(1, t)$ to the identity and eliminating ODE backpropagation, with $\mathcal{L}_{\text{cons}}$ reducing the error of this estimate; (ii) the trajectory-wide integral is replaced by a Monte Carlo estimate at a single $t \sim \text{Uniform}(0, 1)$, the same estimator flow matching uses for training; and (iii) the un-normalized step size $\|g\|$ is replaced by the adaptive scaling $2\lambda\alpha\|u_t\|$ of Eq. 6, removing a known source of instability in DDPG-family methods [3, 44].

3.3 Offline-to-Online Pipeline

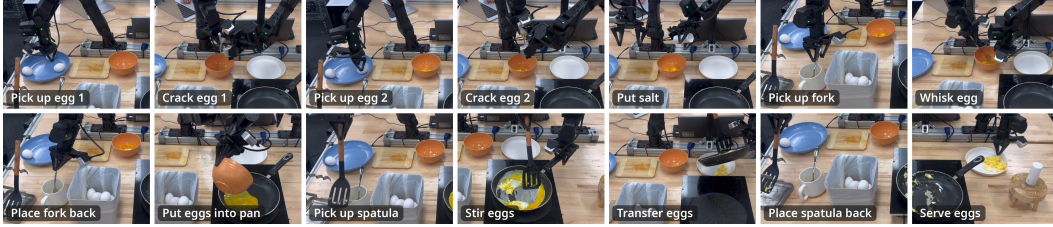
The two components above slot into the pipeline of Figure 2 (middle and right), detailed in Algorithm 1 (Appendix B). In the **offline phase**, the backbone, action head, and value head are jointly trained on demonstrations $\mathcal{D}_{\text{off}}$ with the FlowDPG objective, producing a base policy that initializes online deployment. In the **online phase**, the policy is deployed to collect real-world rollouts that are asynchronously appended to $\mathcal{D}_{\text{on}}$; the learner samples mini-batches at $\sim 1:1$ ratio from $\mathcal{D}_{\text{off}}$ and all rollouts in $\mathcal{D}_{\text{on}}$ to balance demonstration quality and fresh experience. The BC anchor $\mathcal{L}_{\text{BC}}$ is computed only on demonstrations and *successful* rollouts, so it stays tied to successful behavior rather than drifting with the policy. Backbone and reward predictor are frozen in this phase to preserve pre-trained representations and reduce per-step compute; updated actor/critic checkpoints synchronize back to deployment periodically.

4 Experiments

Tasks. We evaluate on two long-horizon tasks on different robots (Figure 3); demonstrations for both are collected with GELLO [46] teleoperation. *AirPods assembly* runs on two Franka arms with randomized case and pod poses and has 8 stages: grasp the case, open it, grasp and insert the right pod, grasp and insert the left pod, close the case, and place it. The policy sees four cameras (front, top, and two wrist views). *Scrambled-egg cooking* runs on two YAM arms and has 14 stages: crack two eggs, add salt, fetch a fork to whisk and return it, pour into a pan, fetch a spatula to stir and



(a) Dual-arm AirPods assembly (Franka), 8 stages



(b) Scrambled-egg cooking (YAM), 14 stages

Figure 3: The two long-horizon tasks, stage by stage (left to right, top to bottom; egg keyframes from one autonomous rollout). A single policy executes all stages end-to-end from raw observations, without stage-level resets or hand-engineered switching.

transfer the eggs, return it, and serve. It adds tool use, fluid handling, and exogenous dynamics, and the policy sees three cameras (top and two wrist views).

Metrics and protocols. The *full sweep* (Table 1) evaluates every method in one round of 25 AirPods cases (50 pod manipulations); failures within a stage do not end the episode, retries count in the per-stage denominators, and **Overall** requires all pods correctly inserted. The *repeated evaluation* (Figure 4) compares BC, QAM, and FlowDPG on both tasks with 50 rollouts per cell, 10 at each of 5 checkpoints of one training run ($\pm$ s.e. over checkpoint means), and adds a **rubric** score grading each stage 0–3 as in $\pi_{0.5}$ [15] (Appendix D). The egg task is evaluated after the offline phase only.

Baselines. We compare against three families of RL methods for adapting a base flow policy. *Value-conditioning* methods use only scalar value/advantage signals: RA-BC [26] reweights the BC objective by SARM progress, AWR [27] performs advantage-exponentially-weighted regression, and RECAP [28] conditions the policy on advantage. *Auxiliary-module* methods keep the base policy frozen and train a separate adapter: DSRL [29] performs RL in the noise space of the denoiser, PLD [30] learns a single-step residual, and RLT [31] attaches an actor-critic head to a compact RL token. The *critic-gradient* category contains the concurrent QAM [5], which converts $\nabla_a Q$ into step-wise supervision via adjoint matching. All baselines share the same backbone, action chunk horizon, reward, value head, and offline data as FlowDPG.

4.1 Main Results

Value-conditioning is bounded by the demonstration support. Reweighting or conditioning over demonstrations cannot synthesize an action outside their distribution: these methods help where the bottleneck is picking the better mode (Open/Close case) but stay near BC on contact-rich stages (Insert R/L pod).

Auxiliary modules leave the base policy untouched. DSRL acts in noise space, separated from the action by the entire ODE; PLD’s single-step residual nudges the endpoint but cannot reshape the denoising; RLT is strongest here, but its MLP actor discards the flow policy’s multimodality. Headroom is capped by the adapter.

Critic-gradient methods update the base policy. FlowDPG uses $\nabla_a Q$ to update the velocity field itself, removing both ceilings. The concurrent QAM [5] pursues the same direction through adjoint

Table 1: Full baseline sweep on AirPods assembly: per-stage, overall, and average success over a single evaluation round of 25 cases (50 pod manipulations); per-stage rates count retries in the denominator; Avg is the mean of the eight per-stage rates. Baselines are trained offline only; Figure 4 gives QAM the same online phase as FlowDPG.

Method	Grasp case	Open case	Grasp R pod	Insert R pod	Grasp L pod	Insert L pod	Close case	Place case	Overall	Avg
BC (base)	100%	78.32%	72.44%	68.54%	78.56%	66.23%	92%	96%	64%	81.51%
<i>Value-conditioning</i>										
RA-BC	100%	86.15%	80.52%	64.52%	82.49%	68.46%	96%	96%	76%	84.27%
AWR	100%	80.65%	78.43%	72.45%	78.77%	70.65%	100%	100%	76%	85.12%
RECAP	100%	89.29%	80.62%	69.69%	76.54%	64.52%	96%	100%	72%	84.58%
<i>Auxiliary-module adaptation</i>										
DSRL	100%	79.85%	75.47%	70.59%	78.13%	67.94%	92%	92%	68%	82.00%
PLD	100%	85.27%	80.13%	79.82%	80.43%	79.59%	96%	96%	76%	87.16%
RLT	100%	86.93%	82.41%	77.92%	84.14%	80.18%	96%	100%	80%	88.45%
<i>Critic-gradient</i>										
QAM	100%	88%	83.33%	70.58%	73.52%	80%	100%	100%	80%	86.93%
FlowDPG (offline)	100%	89.29%	90.63%	84.38%	96.43%	83.33%	100%	100%	88%	93.01%
FlowDPG (+online)	100%	89.29%	92.31%	86.21%	96.67%	88.89%	100%	100%	92%	94.17%

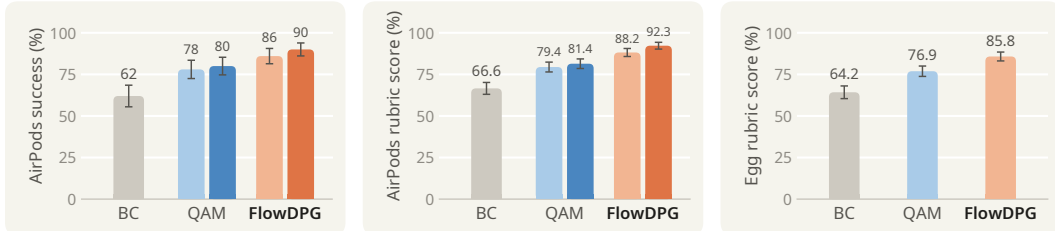


Figure 4: Repeated evaluation against the BC base and the closest baseline QAM. Each bar is 50 rollouts (10 at each of 5 checkpoints of one training run); error bars are the standard error over the five checkpoint means. Lighter shades: after the offline phase; darker shades: after online fine-tuning. The egg task is offline-only. Rubric: 0–3 per stage as a percentage of the available points (Appendix D).

matching but supervises every flow timestep, so critic error at the generated action enters every step’s target; FlowDPG distills a single critic-gradient evaluation via one L2 regression (Sec. 3.2).

Repeated evaluation and a second task. With the same online phase for both methods (Figure 4), FlowDPG leads QAM on AirPods by about ten points in both overall success and rubric score, and online fine-tuning helps FlowDPG more than QAM. Its 90% overall here is consistent with the 92% of the single-round sweep (Table 1). On the egg task we compare only against BC and QAM, which ties for the best baseline on AirPods and is closest to FlowDPG in mechanism; the same ordering holds after the offline phase alone.

4.2 Online RL Improves Robustness to Unseen Disturbances

Online fine-tuning raises FlowDPG’s overall success from 86% to 90% in the repeated evaluation (Figure 4); in the sweep, its per-stage gains all fall on the pod stages (Table 1). To probe generalization beyond the demonstration distribution, we apply three families of disturbances after the policy reaches the relevant stage: (i) *stage undo*—returning a grasped case/pod to the tray or inverting the open/close state of the case; (ii) *in-place adjustment*—dislodging a successfully inserted pod; (iii) *scene perturbation*—reshuffling tray contents while the policy is reaching. Each disturbance is evaluated on 50 trials.

The offline policy handles undo disturbances reasonably (Table 2), since the post-undo observation resembles the start of the corresponding stage; its weak point is scene perturbation, a mid-execution transition absent from demonstrations. Online RL improves all four cases, most on scene perturbation, consistent with deployment-time failures exposing the policy to partial-success states that the critic-guided update learns to recover from.

Table 2: Recovery rates under disturbance families (i)–(iii); Δ in points.

Disturbance	Offline	+Online	Δ
(i) Re-grasp (object replaced in tray)	78%	92%	+14
(i) Re-open/close (case state inverted)	82%	94%	+12
(ii) Re-insert (pod dislodged)	86%	92%	+6
(iii) Adapt grasp (tray reshuffled mid-execution)	70%	88%	+18

Table 3: Critic and action-distribution diagnostics on both tasks.

	AirPods (Franka)	Eggs (YAM)
<i>Critic-gradient informativeness</i>		
Spearman ρ	0.36 ± 0.05	0.28 ± 0.03
<i>Action-distribution changes (before $\rightarrow$ after RL)</i>		
Standardized spread of samples	$1.61 \rightarrow 1.51$ (-0.10 ± 0.03)	$0.87 \rightarrow 0.85$ (-0.02 ± 0.01)
Wasserstein distance to nearby demos	$9.93 \rightarrow 10.02$ ($+0.09 \pm 0.04$)	$7.35 \rightarrow 7.48$ ($+0.13 \pm 0.05$)
Q of policy actions	$32.79 \rightarrow 37.66$ ($+4.87 \pm 1.24$)	$53.84 \rightarrow 59.66$ ($+5.82 \pm 1.54$)
Q of demonstration actions	33.58	54.24

4.3 Diagnostics

A learned critic offers no guarantee outside the data distribution, a limitation shared by DPG-style methods, and we do not explicitly constrain the policy to the demonstration support, since exploring beyond it is part of the purpose of RL. Online, an overvalued action can produce an observable failure, such as a broken eggshell or an improperly seated pod, which SARM registers and the critic learns from, provided SARM remains accurate on newly visited states. Table 3 examines critic-gradient informativeness and action-distribution changes on both tasks.

Critic-gradient informativeness: For each action chunk across 50 BC rollouts, we compute its alignment with the critic gradient, $\cos(\nabla_a Q(s, a), a - \mu_\pi(s))$, where $\mu_\pi(s)$ is the mean of BC’s sampled chunks at s , and its increase in SARM progress. We report their Spearman correlation ($\pm$ s.e. over rollouts). BC was never optimized against Q , so this tests the gradient on actions generated without critic guidance, against observed progress rather than predicted value. The positive correlations on both tasks suggest that the gradient is locally informative within the BC visitation distribution, but do not establish its reliability outside it.

Action-distribution changes: At 80 demonstration states, we sample $K=50$ action chunks before and after RL using the same noise draws. We measure their spread in per-dimension standardized action units, their Wasserstein distance to demonstration actions at nearby states, and their predicted values alongside those of recorded demonstration actions (with paired changes $\pm$ s.e.). All values use the same final critic within each task, isolating policy changes from changes in the scorer. On both tasks, RL modestly narrows the action distribution and slightly increases its distance to nearby demonstrations, while raising predicted policy values above those of demonstration actions. Moving away from demonstrations is compatible with the purpose of RL but does not by itself show improvement, and higher scores from the critic being optimized against cannot separate genuine gains from exploitation of critic errors, which small distributional changes do not rule out. The task-success gains above therefore remain the independent evidence of improvement.

4.4 Ablation: Consistency Regularizer and Adaptive Shift

We ablate each design choice and report both end-to-end success and mechanism-level metrics (Figure 5); all variants are trained with the full online phase. Removing the consistency regularizer lets the projection error $\|\hat{a} - x_1\|$ plateau an order of magnitude higher (Fig. 5(a)), amplifying the Q -value discrepancy $|Q(s, \hat{a}) - Q(s, x_1)|$ (Fig. 5(b)). The critic gradient is then evaluated farther from the data, where it is less reliable, and Overall drops from 92% to 80%. Removing the adaptive shift reduces the update to a raw αg , whose unbounded magnitude makes the training loss oscillate rather than decrease smoothly (Fig. 5(c)) and produces the largest single-component drop (92% to 76%). The two components mitigate the same root cause, a misdirected critic-gradient signal, through complementary means, so removing both adds only a marginal further degradation (72%).

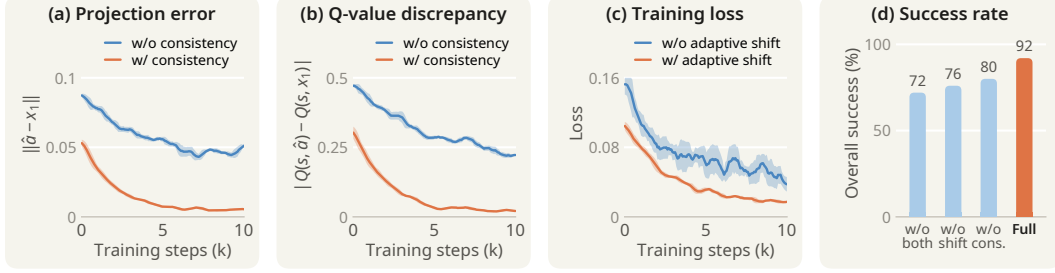


Figure 5: Ablation on the consistency regularizer and adaptive shift. Lines are a moving average, shading ± 1 SD. All variants in (d) include the full online phase.

Full reward	100%	96%	96%	96%	96%	92%	92%	92%
w/o progress	100%	96%	92%	88%	84%	84%	80%	80%
w/o stage transition	100%	96%	92%	88%	84%	80%	76%	76%
Only terminal	100%	80%	76%	72%	68%	64%	60%	60%
	Grasp case	Open case	Grasp R pod	Insert R pod	Grasp L pod	Insert L pod	Close case	Place case

Figure 6: Cumulative stage coverage on AirPods under reward ablation: the fraction of episodes that complete every stage up to and including each column.

4.5 Ablation: Reward Components

We compare the full composite reward in Eq. 3 against three variants; Figure 6 reports cumulative stage coverage with a distinct failure signature per variant. Without the progress term, the reward is constant within each stage except at transitions, so within-stage differences in Q come only from discounting, $\nabla_a Q$ is weak, and coverage erodes gradually across the sequence (80%). Removing the stage-transition term keeps the dense progress shaping but eliminates the discrete bonus at sub-task completion, so $\nabla_a Q$ does not strongly favor closing out a sub-task over hovering near its end (76%). Removing both leaves only the terminal bonus: two visually similar states, the gripper holding a closed case before opening and again after the case is re-closed, become indistinguishable under reward. The policy frequently bypasses the entire pod-handling sequence to place the case directly, dropping sharply at Open case (96% $\rightarrow$ 80%) and ending at 60%, below even the BC base. The progress and stage-transition terms are thus not redundant: with such sparse shaping, the distilled critic-gradient signal carries more noise than information.

5 Conclusion and Limitations

We presented **FlowDPG**, a DDPG-style method for flow matching policies that distills the critic gradient into the velocity field via a single-step projection and L2 regression, avoiding backpropagation through the denoising ODE and recovering vanilla DPG under three explicit approximations. On two long-horizon real-world tasks, it attains 92% end-to-end success on dual-arm AirPods assembly and an 85.8% rubric score on 14-stage egg cooking.

Limitations. FlowDPG relies on a learned critic, whose gradient is most reliable near the data. The one-step estimate and the adaptive shift reduce, but do not eliminate, the effect of critic errors, and online interaction corrects them only as far as the reward model scores newly visited states accurately. Our reward assumes tasks that decompose into stages with a dense progress signal, and the BC anchor that keeps actions feasible may also limit how far the policy can improve. Finally, our evaluation covers two long-horizon tasks on two robot platforms; extending FlowDPG to larger VLA policies and a broader range of tasks is left for future work.

Acknowledgments

We thank Ananye Agarwal and Kevin Gmelin for helpful discussions. We are grateful to the Skild AI manipulation and hardware teams for their support, and to all the contractors who collected and annotated the data used in this work.

References

- [1] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- [2] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.
- [3] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2016.
- [4] B. Kang, X. Ma, C. Du, T. Pang, and S. Yan. Efficient diffusion policies for offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [5] Q. Li and S. Levine. Q-learning with adjoint matching. *arXiv preprint arXiv:2601.14234*, 2026.
- [6] Open X-Embodiment Collaboration, A. O’Neill, A. Rehman, A. Gupta, et al. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. *arXiv preprint arXiv:2310.08864*, 2023.
- [7] F. Ebert, Y. Yang, K. Schmeckpeper, B. Bucher, G. Georgakis, K. Daniilidis, C. Finn, and S. Levine. Bridge Data: Boosting Generalization of Robotic Skills with Cross-Domain Datasets. *arXiv preprint arXiv:2109.13396*, 2021.
- [8] S. Dasari, F. Ebert, S. Tian, S. Nair, B. Bucher, K. Schmeckpeper, S. Singh, S. Levine, and C. Finn. RoboNet: Large-scale multi-robot learning. *arXiv preprint arXiv:1910.11215*, 2019.
- [9] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, et al. DROID: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- [10] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems (RSS)*, 2023.
- [11] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.
- [12] S. Dasari and A. Gupta. Transformers for one-shot visual imitation. In *Conference on Robot Learning*, pages 2071–2084. PMLR, 2021.
- [13] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [14] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [15] Physical Intelligence, K. Black, N. Brown, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

- [16] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [17] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, et al. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [18] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*, 2023.
- [19] X. Chen, J. Djolonga, P. Padlewski, B. Mustafa, S. Changpinyo, J. Wu, C. R. Ruiz, S. Goodman, X. Wang, Y. Tay, et al. PaLI-X: On Scaling Up a Multilingual Vision and Language Model. *arXiv preprint arXiv:2305.18565*, 2023.
- [20] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence. PaLM-E: An Embodied Multimodal Language Model. *arXiv preprint arXiv:2303.03378*, 2023.
- [21] S. Karamcheti, S. Nair, A. Balakrishna, P. Liang, T. Kollar, and D. Sadigh. Prismatic VLMs: Investigating the Design Space of Visually-Conditioned Language Models. In *International Conference on Machine Learning*, 2024.
- [22] L. Beyer, A. Steiner, A. Susano Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Alabdulmohsin, M. Tschannen, E. Bugliarello, T. Unterthiner, D. Keysers, S. Koppula, F. Liu, A. Grycner, A. Gritsenko, N. Houlsby, M. Kumar, K. Rong, J. Eisenschlos, R. Kabra, M. Bauer, M. Bošnjak, X. Chen, M. Minderer, P. Voigtlaender, I. Bica, I. Balazevic, J. Puigcerver, P. Papalampidi, O. Henaff, X. Xiong, R. Soricut, J. Harmsen, and X. Zhai. PaliGemma: A versatile 3B VLM for transfer. *arXiv preprint arXiv:2407.07726*, 2024.
- [23] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. In *Robotics: Science and Systems*, 2023.
- [24] M. Reuss, M. Li, X. Jia, and R. Lioutikov. Goal-conditioned imitation learning using score-based diffusion policies. In *Robotics: Science and Systems (RSS)*, 2023.
- [25] J. Barreiros, A. Beaulieu, A. Bhat, R. Cory, E. Cousineau, H. Dai, C.-H. Fang, K. Hashimoto, M. Z. Irshad, M. Itkina, et al. A careful examination of large behavior models for multitask dexterous manipulation. *Science Robotics*, 11(113):eadp6201, 2026.
- [26] Q. Chen, J. Yu, M. Schwager, P. Abbeel, Y. Shentu, and P. Wu. SARM: Stage-aware reward modeling for long horizon robot manipulation. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2509.25358>.
- [27] X. B. Peng, A. Kumar, G. Zhang, and S. Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.
- [28] Physical Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, et al. $\pi_{0.6}^*$: A VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.
- [29] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2025.
- [30] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, G. Shi, and Y. Zhu. Self-improving vision-language-action models with data generation via residual RL. *arXiv preprint arXiv:2511.00091*, 2025.

- [31] C. Xu, J. T. Springenberg, M. Equi, A. Amin, A. Esmail, S. Levine, and L. Ke. RL token: Bootstrapping online RL with vision-language-action models. *arXiv preprint arXiv:2604.23073*, 2026.
- [32] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma. Learning a diffusion model policy from rewards via Q-score matching. In *International Conference on Machine Learning (ICML)*, 2024.
- [33] Y. Wang, X. Li, P. Xie, P. Yang, B. Nie, Y. Cai, Q. Zhang, C. Qu, J. Wu, J. Song, X. Ren, J. Huang, M. Pan, S. Feng, Z. Chen, and J. Luo. Learning while deploying: Fleet-scale reinforcement learning for generalist robot policies. *arXiv preprint arXiv:2605.00416*, 2026.
- [34] Z. Wang, J. J. Hunt, and M. Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2023.
- [35] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz. Diffusion policy policy optimization. In *International Conference on Learning Representations (ICLR)*, 2025.
- [36] T. Zhang, C. Yu, S. Su, and Y. Wang. ReinFlow: Fine-tuning flow matching policy with online reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [37] J. Liu, G. Liu, J. Liang, Y. Li, J. Liu, X. Wang, P. Wan, D. Zhang, and W. Ouyang. Flow-GRPO: Training flow matching models via online RL. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [38] S. Park, Q. Li, and S. Levine. Flow Q-learning. In *International Conference on Machine Learning (ICML)*, 2025. URL <https://arxiv.org/abs/2502.02538>.
- [39] Q. Li, Z. Zhou, and S. Levine. Reinforcement learning with action chunking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2507.07969>.
- [40] P. Koirala and C. Fleming. Flow-based single-step completion for efficient and expressive policy learning. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2506.21427>.
- [41] T. X. Nguyen and C. D. Yoo. One-step flow Q-learning: Addressing the diffusion policy bottleneck in offline reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2508.13904>.
- [42] A. Houssaini, M. H. Danesh, A. Abyaneh, S. Fujimoto, H.-C. Lin, and D. Meger. Drift Q-learning. *arXiv preprint arXiv:2606.00350*, 2026. URL <https://arxiv.org/abs/2606.00350>.
- [43] I. Kostrikov, A. Nair, and S. Levine. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2110.06169>.
- [44] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*, pages 1587–1596. PMLR, 2018. URL <https://arxiv.org/abs/1802.09477>.
- [45] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *International Conference on Machine Learning (ICML)*, pages 387–395. PMLR, 2014.

- [46] P. Wu, Y. Shentu, Z. Yi, X. Lin, and P. Abbeel. GELLO: A general, low-cost, and intuitive teleoperation framework for robot manipulators. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.

A Derivation of the Vanilla DPG Gradient on a Flow Matching Policy

We provide the full derivation of Eq. 10 for completeness. The deterministic actor is the ODE terminus $x_1 = x_0 + \int_0^1 v_\theta(x_t, t, s) dt$, with $x_0 \sim \mathcal{N}(0, I)$ independent of θ . The DDPG actor loss is $\mathcal{L}_{\text{DPG}} = -Q(s, x_1)$, so by the chain rule

$$\nabla_\theta \mathcal{L}_{\text{DPG}} = -\nabla_a Q(s, x_1) \cdot \nabla_\theta x_1 = -g^\top \cdot \nabla_\theta x_1, \quad (12)$$

where $g := \nabla_a Q(s, a)|_{a=x_1} \in \mathbb{R}^{|a|}$ is the critic gradient at the action terminus, treated throughout as a column vector. It remains to derive $\nabla_\theta x_1$ for the ODE actor.

Forward sensitivity equation. Define the sensitivity matrix $M(t) := \partial x_t / \partial \theta \in \mathbb{R}^{|a| \times |\theta|}$, with initial condition $M(0) = 0$ since x_0 is parameter-free. Differentiating the ODE $\frac{dx_t}{dt} = v_\theta(x_t, t, s)$ with respect to θ and applying the chain rule on the right-hand side,

$$\frac{dM(t)}{dt} = \nabla_x v_\theta(x_t, t, s) \cdot M(t) + \nabla_\theta v_\theta(x_t, t, s). \quad (13)$$

This is a linear non-homogeneous ODE in $M(t)$. By variation of parameters, its solution at $t=1$ is

$$M(1) = \int_0^1 \Phi(1, t) \cdot \nabla_\theta v_\theta(x_t, t, s) dt, \quad (14)$$

where $\Phi(\sigma, t)$ is the state-transition matrix associated with the homogeneous part of Eq. 13, defined as the solution to

$$\frac{\partial \Phi(\sigma, t)}{\partial \sigma} = \nabla_x v_\theta(x_\sigma, \sigma, s) \cdot \Phi(\sigma, t), \quad \Phi(t, t) = I. \quad (15)$$

$\Phi(1, t)$ describes how a perturbation of x_t propagates to a perturbation of x_1 — i.e., the Jacobian $\partial x_1 / \partial x_t$.

Final form. Substituting Eq. 14 into Eq. 12 gives the vanilla DPG gradient quoted in the main text:

$$\nabla_\theta \mathcal{L}_{\text{DPG}} = - \int_0^1 g^\top \Phi(1, t) \cdot \nabla_\theta v_\theta(x_t, t, s) dt. \quad (16)$$

Discrete-time form. Under Euler discretization $x_{t_{i+1}} = x_{t_i} + \Delta t \cdot v_\theta(x_{t_i}, t_i, s)$ with T steps and $\Delta t = 1/T$, the chain-rule recursion gives

$$\nabla_\theta x_1 = \Delta t \cdot \sum_{i=0}^{T-1} \left[\prod_{j=i+1}^{T-1} J_j \right] \cdot \nabla_\theta v_\theta(x_{t_i}, t_i, s), \quad J_j := I + \Delta t \cdot \nabla_x v_\theta(x_{t_j}, t_j, s), \quad (17)$$

with the convention $\prod_{j=T}^{T-1} (\cdot) = I$, so that the term $i = T-1$ contributes only $\nabla_\theta v_\theta(x_{t_{T-1}}, \cdot)$.

The matrix product $\prod_{j=i+1}^{T-1} J_j$ is the discrete analogue of $\Phi(1, t_i)$. In practice this recursion is what an autograd implementation would compute when backpropagating through the ODE solver. The product structure is precisely the source of the gradient explosion/vanishing pathology: small deviations in the spectral radius of J_j from unity compound multiplicatively across the T steps, making the update numerically fragile when v_θ is not yet well-trained.

Adjoint form. Equivalently, define the adjoint state $\lambda(t) := \Phi(1, t)^\top g \in \mathbb{R}^{|a|}$, which propagates the terminal gradient g backward in time. It satisfies the linear backward ODE

$$\frac{d\lambda(t)}{dt} = -\nabla_x v_\theta(x_t, t, s)^\top \cdot \lambda(t), \quad \lambda(1) = g, \quad (18)$$

integrated from $t = 1$ to $t = 0$. The DPG gradient can then be written as

$$\nabla_\theta \mathcal{L}_{\text{DPG}} = - \int_0^1 \lambda(t)^\top \cdot \nabla_\theta v_\theta(x_t, t, s) dt. \quad (19)$$

This is the form that adjoint methods solve directly, computing $\lambda(t)$ by backward integration and accumulating ∇_θ contributions at each t . FlowDPG bypasses both the forward sensitivity matrix Φ and the adjoint state λ entirely, instead distilling a single critic gradient evaluated at the in-distribution projected clean action $\hat{a}$.

B Offline-to-Online Training Algorithm

Algorithm 1 summarizes the full offline-to-online procedure of Sec. 3.3.

Algorithm 1 FlowDPG: Offline-to-Online Training

Require: Offline buffer $\mathcal{D}_{\text{off}}$; online buffer $\mathcal{D}_{\text{on}}$ (initially empty); guidance scale α ; consistency weight μ_{cons} ; max distillation weight λ_{max} ; warmup N_{warmup} ; EMA rate ρ ; expectile τ .

- 1: Pretrain SARM on a small stage-annotated subset of the data
 - 2: Initialize backbone, action head v_θ , value head $(V_\psi, Q_{\phi_1}, Q_{\phi_2})$; EMA targets $Q_{\tilde{\phi}_i} \leftarrow Q_{\phi_i}$
-
- Offline phase**
-
- 3: **for** $k = 1, \dots, N_{\text{off}}$ **do**
 - 4: Sample mini-batch from $\mathcal{D}_{\text{off}}$; compute r_t from the reward predictor
 - 5: Sample $x_0 \sim \mathcal{N}(0, I)$, $t \sim \text{Uniform}(0, 1)$; form x_t, u_t
 - 6: Compute $v_t = v_\theta(x_t, t, s)$ and $\hat{a}$ via Eq. 4
 - 7: $\mathcal{L}_{\text{cons}} \leftarrow \|\hat{a} - x_1\|^2$
 - 8: $g \leftarrow \nabla_{\hat{a}} \min_i Q_{\phi_i}(s, \hat{a})$ with stop-grad on critic
 - 9: $\Delta \leftarrow \alpha \cdot \|u_t\| / (\|g\| + \varepsilon) \cdot g$
 - 10: $a^* \leftarrow \hat{a} + \Delta$; $u_t^* \leftarrow a^* - x_0$
 - 11: $\lambda \leftarrow \lambda_{\text{max}} \cdot \min(k/N_{\text{warmup}}, 1)$
 - 12: Update θ by minimizing $\mathcal{L}_{\text{FlowDPG}} + \mu_{\text{cons}} \mathcal{L}_{\text{cons}}$ via Eqs. 8, 9
 - 13: Update ψ, ϕ_1, ϕ_2 via Eqs. 1–2
 - 14: EMA targets: $\phi_i \leftarrow \rho \phi_i + (1 - \rho) \phi_i$
 - 15: **end for**
-
- Online phase**
-
- 16: Deploy π_θ for autonomous rollout collection
 - 17: **for** $k = 1, \dots, N_{\text{on}}$ **do**
 - 18: Asynchronously append rollouts to $\mathcal{D}_{\text{on}}$ with rewards from the reward predictor
 - 19: Sample mini-batch from $\mathcal{D}_{\text{off}} \cup \mathcal{D}_{\text{on}}$ at $\sim 1:1$ ratio; BC anchor on demos and successful rollouts only
 - 20: Update $\theta, \psi, \phi_1, \phi_2$ as in the offline phase, with backbone and SARM frozen
 - 21: **if** $k \bmod N_{\text{sync}} = 0$ **then**
 - 22: Synchronize π_θ to deployment
 - 23: **end if**
 - 24: **end for**
 - 25: **return** v_θ
-

C SARM Signals and Value Estimates Along a Rollout

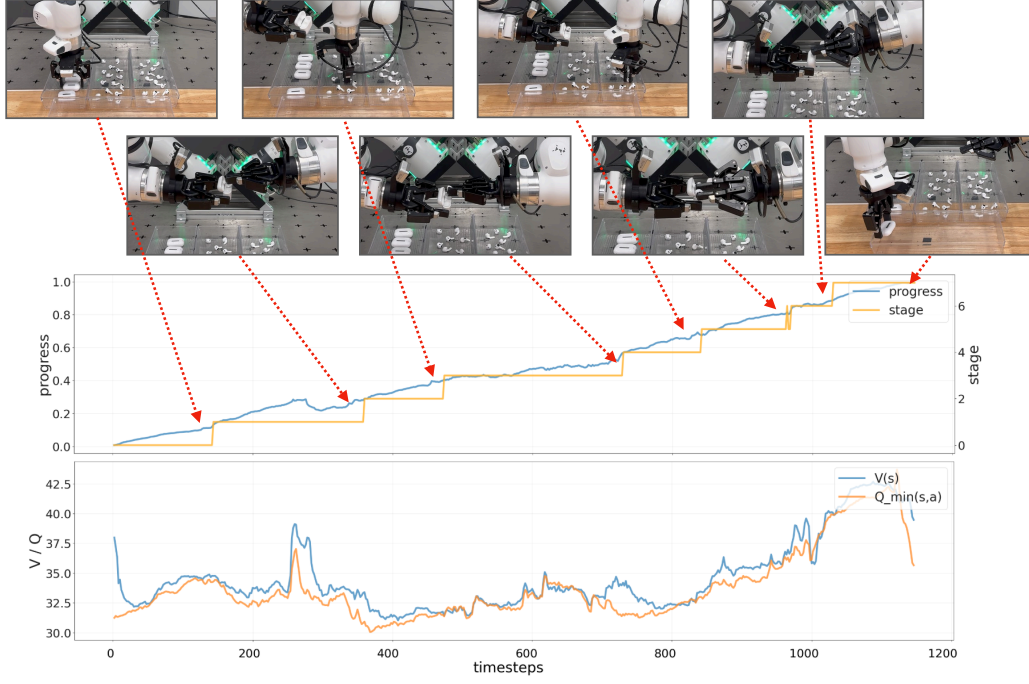


Figure 7: SARM signals and value estimates along a successful AirPods assembly rollout. **Top:** keyframes at SARM stage transitions. **Middle:** predicted within-stage progress $\phi(o_t)$ (blue) and stage label $z(o_t)$ (yellow step). **Bottom:** learned value $V_\psi(s_t)$ (blue) and twin-critic minimum $\min_i Q_{\phi_i}(s_t, a_t)$ (orange) over the same rollout.

Figure 7 visualizes how the SARM signals and the learned value/Q-estimates evolve along a successful real-world execution. The progress curve $\phi(o_t)$ provides a meaningful per-step signal even within a single stage, supplying the dense shaping that FlowDPG distills via the critic, and stage-label jumps align with the keyframes in the top strip, illustrating when the stage-transition bonus in Eq. 3 fires. The value and Q-curves track each other closely throughout the rollout ($V_\psi \approx \min_i Q_{\phi_i}$), indicating that the value and critic heads are mutually consistent. The value trends upward over the rollout, mostly in the later stages, with transient dips at contact-rich stages that coincide with short recoveries from minor failures.

D Evaluation Rubrics

Each stage is scored 0–3; a stage that is never reached scores 0. The criteria below were fixed before scoring. The rubric score is the total as a percentage of the available points: 24 for AirPods assembly (8 stages) and 42 for scrambled-egg cooking (14 stages). In the egg task, *pick up egg* and *crack egg* occur twice, once per egg, giving 14 scored stages from the 12 stage types in Table 5.

Table 4: Rubric for AirPods assembly (Franka), 8 stages, 24 points.

Stage	3 – Successful	2 – Minor error	1 – Major error	0 – Failed
Grasp case	Secure grasp, correct orientation, no slip	Grasp with adjustment or delay	Case lifted but shifted or dropped once	Fails to grasp case
Open case	Lid fully opened in one clean motion	Opens with adjustment or hesitation	Opens only after repeated attempts, or case displaced	Lid not opened
Grasp right pod	Secure grasp, correct approach direction	Grasp with adjustment or delay	Wrong approach direction, pod dislodged then re-grasped	Fails to grasp pod
Insert right pod	Seated on the first attempt, fully in	Seated after one retry	Seated after two or more retries, or insertion dislodges the other pod	Pod not seated
Grasp left pod	Secure grasp, correct approach direction	Grasp with adjustment or delay	Wrong approach direction, pod dislodged then re-grasped	Fails to grasp pod
Insert left pod	Seated on the first attempt, fully in	Seated after one retry	Seated after two or more retries, or insertion dislodges the right pod	Pod not seated
Close case	Lid closed cleanly with both pods seated	Closes after minor adjustment	Closes with a pod unseated, or needs repositioning	Case not closed
Place case	Placed upright within the target region	Placed with minor misalignment	Dropped or placed outside the region	Case not placed

Table 5: Rubric for scrambled-egg cooking (YAM), 14 stages, 42 points.

Stage	3 – Successful	2 – Minor error	1 – Major error	0 – Failed
Pick up egg	Secure grasp, no slip or damage	Grasp with adjustment or delay	Lifts egg but drops or damages it	Fails to grasp egg
Crack egg	Clean crack, proper shell separation	Minor shell fragments or awkward motion	Incorrect or messy crack	Fails to crack egg
Put salt	Correct amount applied accurately	Minor spillage or hesitation	Incorrect amount or placement	Salt not applied, or bottle dropped
Pick up fork	Secure and correct grasp	Grasp with adjustment	Unstable or incorrect grasp	Fails to pick up fork, or drops it
Whisk egg	Thorough whisking with correct motion	Partial or inefficient whisking	Minimal or incorrect motion	No whisking
Place fork back	Placed neatly in correct location	Minor misalignment	Dropped or poorly placed	Fork not placed back
Put eggs into pan	Clean transfer into pan	Minor spillage or hesitation	Significant spillage	Eggs not transferred
Pick up spatula	Secure and correct grasp	Grasp with adjustment	Unstable or incorrect grasp	Fails to pick up spatula
Stir eggs	Proper stirring across pan	Incomplete or uneven stirring	Minimal or incorrect stirring	No stirring
Transfer eggs	Clean and accurate transfer	Minor spillage or inefficiency	Partial or messy transfer	Eggs not transferred
Place spatula back	Placed correctly and safely	Minor misplacement	Dropped or unsafe placement	Spatula not placed back
Serve eggs	Served correctly and fully	Minor error or delay	Incorrect or incomplete serving	Eggs not served